

isolated-vm: From Type Confusion to Sandbox Escape and Potential RCE

The vulnerability ("GHSA-864f-rcv7-6rh4"), which has yet to be assigned a CVE identifier, impacts all versions of the library before and including 7.0.0. It has been patched in versions 6.2.0 and 7.0.1 released earlier this month(August).

`isolated-vm` is a `Node.js` library designed to execute untrusted JavaScript code inside isolated V8 Isolates. The main purpose of this isolation mechanism is to prevent potentially malicious JavaScript code from accessing the host application's resources or executing code outside of its assigned sandbox.

A recently discovered vulnerability in `isolated-vm` demonstrates how a flaw in the communication layer between the sandbox and the host can potentially break this security boundary. The vulnerability is related to improper type handling in the `ExternalCopy` mechanism and can lead to type confusion, memory corruption, sandbox escape, and potentially remote code execution (RCE).

The main security purpose of a sandbox is to isolate untrusted code from the host environment. If an attacker can bypass this isolation, the security guarantees provided by the sandbox can effectively be defeated.

Type confusion via a TOCTOU gap in transferList

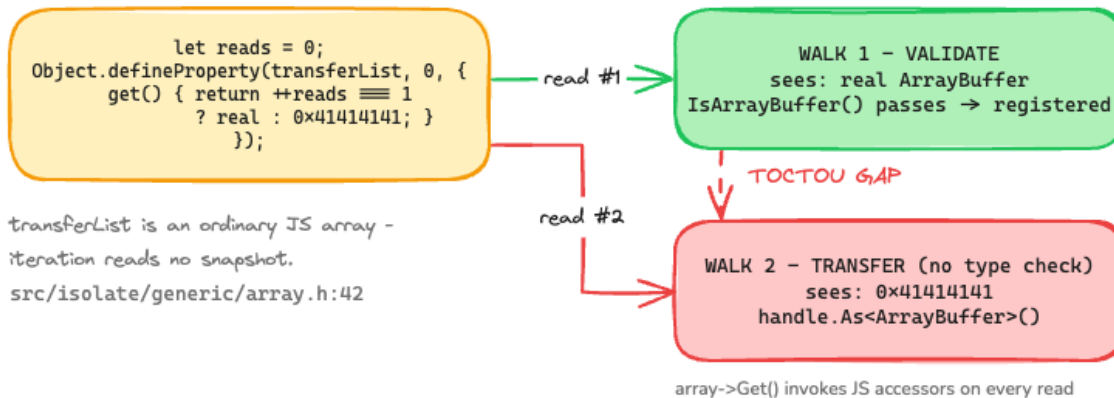
GHSA-864f-rcv7-6rh4 - isolated-vm ≤ 7.0.0

STEP 1 - REACHABILITY FROM INSIDE THE SANDBOX

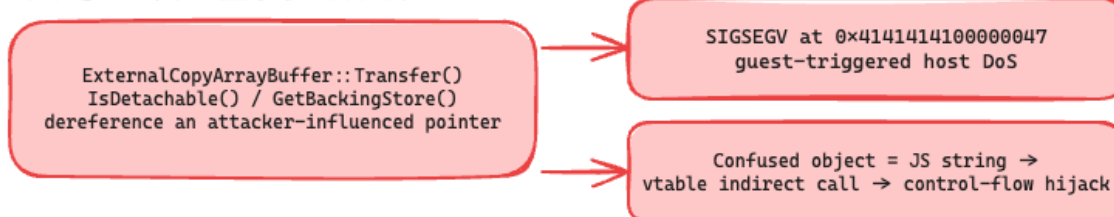


The ExternalCopy constructor crosses the boundary as a live, callable class

STEP 2 - ONE ARRAY, TWO WALKS



STEP 3 - CONTROLLED DEREERENCE

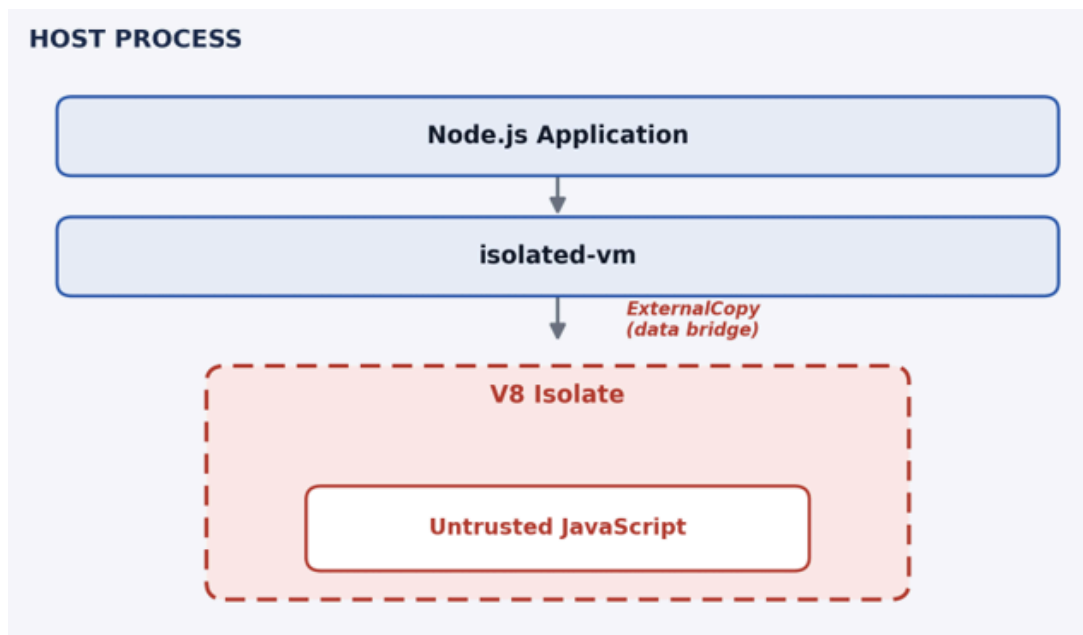


Fixed in 7.0.1 / 6.2.0 - DisallowJavascriptExecutionScope blocks getters during the copy

Sandbox Boundary

When untrusted JavaScript code is executed, it runs inside a separate V8 Isolate rather than directly inside the main Node.js process. The untrusted JavaScript should remain inside the isolated environment and should not be able to directly access the memory or control the execution flow of the host process.

However, **the sandbox and host sometimes need to exchange data**. This is where components such as `ExternalCopy` become important.

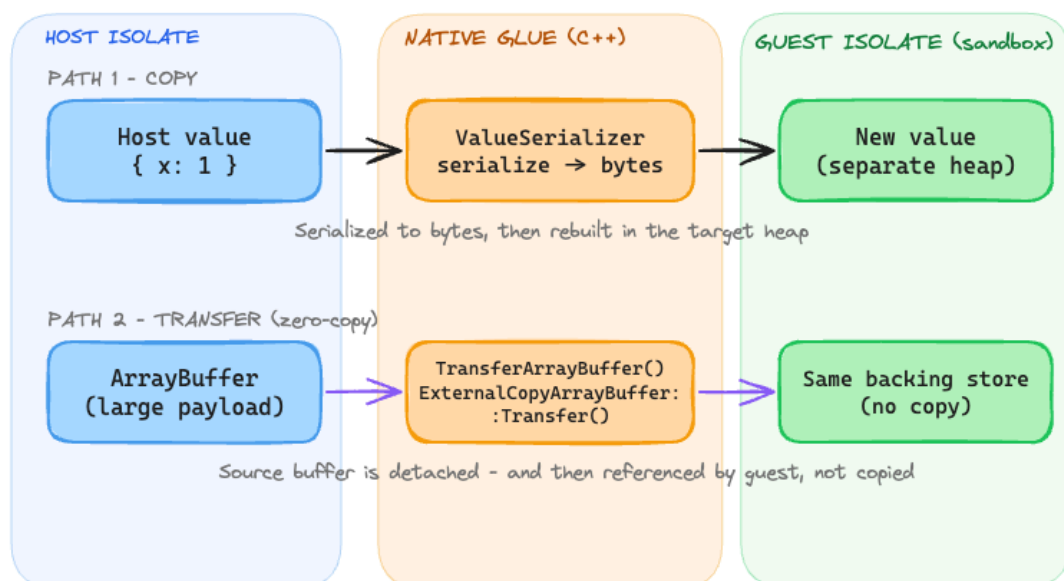


isolated-vm runs untrusted JavaScript inside a V8 Isolate, nested within the host Node.js process.

ExternalCopy is used to transfer JavaScript values between the isolated environment and the host environment. Because this communication involves different environments and native C++ code, the data must be correctly interpreted and represented in memory. A vulnerability in this process can therefore have serious security consequences.

ExternalCopy: crossing the isolate boundary

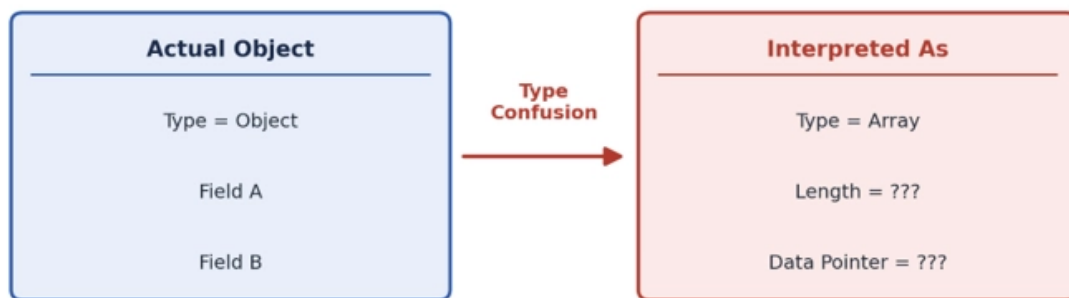
```
new ExternalCopy(value, { transferList: [buf] })
```



How ExternalCopy crosses the isolate boundary

Type Confusion

The main technical issue is related to type confusion — this occurs when a program interprets an object as one data type even though the object actually belongs to another type. For example, a program may expect an object with a `Type`, `Length`, and `Data Pointer`, as with an array. If an object of another type is incorrectly interpreted as an array, the same memory locations may be treated as completely different values.



The host code reads “Field A” and “Field B” as if they were a Length and a Data Pointer, using attacker-influenced values for a subsequent memory operation.

Type confusion becomes particularly dangerous when the incorrectly interpreted data is used for memory operations. For example, if a value is incorrectly interpreted as a pointer, the program may attempt to access an unintended memory address:

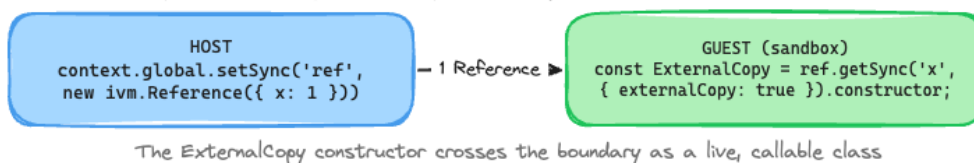
- Incorrect type interpretation
- Incorrect memory layout
- Invalid pointer
- Invalid read / write
- Memory corruption

Memory corruption means that the program reads from or writes to memory in an unintended way. If an attacker can influence where or how this memory operation occurs, they may be able to modify important data belonging to the host process.

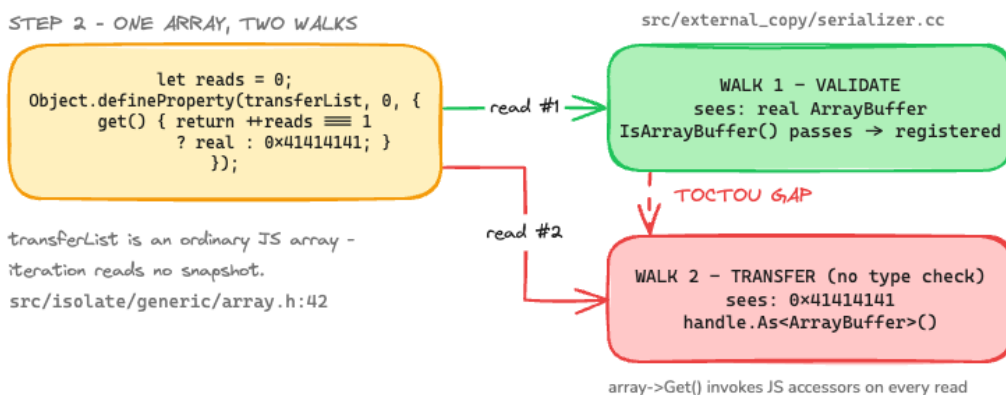
Type confusion via a TOCTOU gap in transferList

GHSA-864f-rcv7-6rh4 - isolated-vm ≤ 7.0.0

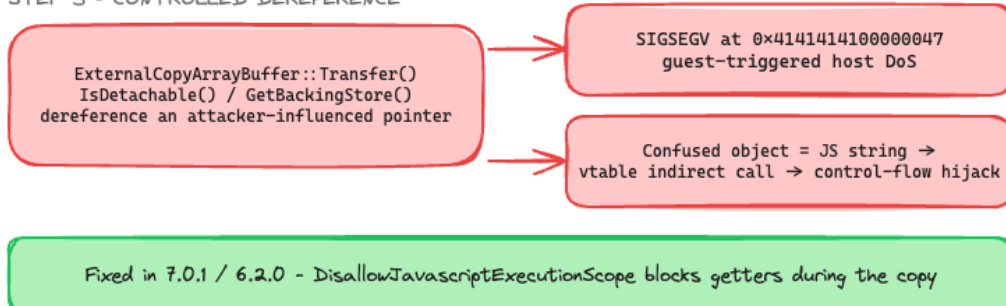
STEP 1 - REACHABILITY FROM INSIDE THE SANDBOX



STEP 2 - ONE ARRAY, TWO WALKS



STEP 3 - CONTROLLED DEREERENCE



A program normally follows a legitimate execution path - Function A, then Function B, then Function C. If a control-flow-related value is corrupted, the execution path may potentially be redirected toward an attacker-controlled location.



It is important to distinguish between memory corruption and guaranteed RCE. Memory corruption alone does not automatically mean that an attacker can execute arbitrary code. Additional exploitation primitives and conditions may be required.

Sandbox Escape and Potential RCE

The most important security impact of this vulnerability is the possibility of escaping the JavaScript sandbox. The sandbox is specifically designed to prevent untrusted code from crossing this boundary. If the vulnerability allows an attacker to

corrupt memory belonging to the host process, that boundary can potentially be crossed.

If the attacker is able to successfully manipulate the host process's memory and control its execution flow, the final impact may be arbitrary code execution on the host. The attacker does not necessarily need direct access to the host system at the beginning — instead, the attacker starts with code execution inside the sandbox and attempts to use the vulnerability to cross the isolation boundary.

The Fix

The maintainer responded quickly and shipped a fix in versions 7.0.1 and 6.2.0. The patch wraps `ExternalCopy::Copy` in a `v8::Isolate::DisallowJavascriptExecutionScope`, which prevents *any* user JavaScript (getters, proxies, interceptors) from running during the copy. That is the shared prerequisite for the type confusion (the `transferList` getter). Users on any affected version should upgrade immediately.

By preventing JavaScript execution during this sensitive operation, the attacker can no longer use JavaScript execution to modify the object between the relevant check and subsequent use. This removes the shared prerequisite that enabled the type confusion.

An important lesson from this vulnerability is that the underlying V8 isolation mechanism was not itself broken. The V8 Isolate continued to provide the intended memory separation between contexts. The problem existed in the additional native code responsible for communicating across the boundary. Therefore, the vulnerability should be understood as a boundary-layer vulnerability, rather than a fundamental failure of V8's isolation model.

Mitigations and Takeaways

1. **Upgrade now.** Move to isolated-vm 7.0.1 (or 6.2.0 on the 6.x line). The fix is small and directly closes both the type confusion and the timeout bypass.
2. **Treat every capability you share into a sandbox as an attack surface.** A single `Reference` was enough to reach the vulnerable constructor. Share the absolute minimum, and assume anything you expose can be turned back on you.
3. **Scrutinize the glue, not just the boundary.** A correct isolation model does not make a library safe on its own. The marshaling layer, especially when it is native code handling attacker-controlled objects, deserves the same scrutiny as the primitive it wraps.

4. **Know your dependencies' real security posture with reachability.** Whether this bug is exploitable for you depends on whether untrusted code reaches `ExternalCopy` with attacker-influenced input. Software composition analysis with reachability lets you answer that instead of guessing.

References

- <https://thehackernews.com/2026/08/isolated-vm-flaw-lets-sandboxed.html>
- <https://www.endorlabs.com/learn/ghsa-864f-rcv7-6rh4-critical-type-confusion-vulnerability-in-isolated-vm>
- <https://devops.com/critical-flaw-in-isolated-vm-can-lead-to-sandbox-escape-rce-threat/>